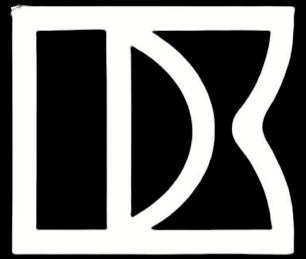
GIT & GITHUB:
A JORNADA DO CÓDIGO

Fundamentos e Fluxo de Trabalho



Data Lab - Treinamento

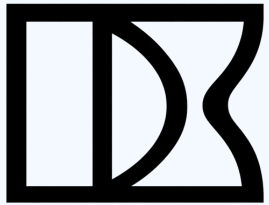
Sumário

01 **Controle de versão**
VCM, VCS, SCM e o papel do Git

02 **Git e GitHub**
Conceitos, fluxo de trabalho e comandos essenciais

03 **Instalação e configuração**
Prepare o Git e configure o acesso ao GitHub

04 **Exercícios**
Aplique o fluxo completo em um repositório



Controle de Versão

VCM, VCS, SCM E GIT

Guarde assim: VCM é o guarda-chuva; VCS é o tipo de ferramenta; SCM é o uso voltado ao código-fonte.

VCM | Version Control Management

Gerenciamento amplo das versões de um produto ou componente.

VCS | Version Control System

Sistema/ferramenta que registra, compara e recupera versões.

SCM | Source Code Management

Aplicação do controle de versões ao código-fonte. API Python, site HTML/CSS, scripts SQL.

MODELOS DE VCS (VERSION CONTROL SYSTEM)



VCS Centralizado

Um único servidor central contém o histórico.

Usuários baixam apenas a versão atual.

Se o servidor falha, o histórico pode ser perdido.

Exemplos: SVN, CVS.



VCS Distribuído

Cada usuário tem uma cópia completa do histórico.

Trabalho offline é possível e rápido.

Maior segurança: cada computador é um backup.

Exemplos: Git, Mercurial.

O QUE É GIT?

O Git é um sistema de controle de versão distribuído, criado por Linus Torvalds em 2005. Ele funciona como uma "máquina do tempo" para o seu código, permitindo registrar cada alteração feita.

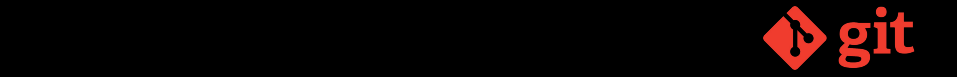
Velocidade
Operações locais são praticamente instantâneas.

Integridade
Garante que o histórico de alterações não seja corrompido.

Ramificação
Permite trabalhar em várias versões do projeto simultaneamente.



O QUE É GIT?



“Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.”

“Git is lightning fast and has a huge ecosystem of GUIs, hosting services, and command-line tools.”

Git = um VCS distribuído (DVCS). Quando usado para código-fonte, ele funciona como ferramenta de SCM.
Fonte: site oficial do Git (git-scm.com)

ARQUITETURA DO GIT: AS TRÊS ÁREAS

Working Directory

Sua pasta local onde você edita os arquivos diretamente.

→

Staging Area

O arquivo de índice que lista o que irá no próximo commit.

→

Local Repo (.git)

Onde o Git armazena os metadados e o banco de dados de objetos.

Fluxo: Checkout do projeto → Modificação de arquivos → Stage dos arquivos → Commit

CICLO DE VIDA DOS ARQUIVOS

Untracked (Não rastreados)

Arquivos que NÃO estão dentro do Git, apenas no disco rígido. O Git não tem registro de sua existência.

Tracked (Rastreados)

O arquivo já foi adicionado a um repositório e o Git monitora suas mudanças.

Staged (Em etapas)

O arquivo foi adicionado à área de preparação, mas ainda não foi comprometido (commit).

Modified (Modificados)

O arquivo foi alterado na sua mesa de trabalho, mas a mudança ainda não foi salva no Git.

Unmodified (Não modificados)

O arquivo não sofreu alterações desde o último commit.

Committed (Comprometidos)

O arquivo já foi atualizado e salvo permanentemente no banco de dados do repositório local.

O QUE É GITHUB?

O GitHub é uma plataforma de hospedagem de código-fonte e arquivos que utiliza o Git. É o maior repositório de software do mundo, facilitando a colaboração global.



Hospedagem na Nuvem

Seus projetos acessíveis de qualquer lugar do mundo.



Colaboração

Pull Requests e Issues para gestão eficiente de projetos.



Comunidade

Perfil de desenvolvedor e portfólio de projetos open-source.

ALÉM DO GITHUB

Existem outras plataformas excelentes que utilizam Git para colaboração:

GitLab

Foco em DevOps e CI/CD integrado. Muito popular para instalações em servidores privados (On-premise).

Bitbucket

Integração nativa com o ecossistema Atlassian (Jira, Trello). Ideal para fluxos de trabalho corporativos.

Azure DevOps

A solução completa da Microsoft para o ciclo de vida de software, incluindo repositórios Git robustos.

Gitea / Gogs

Opções extremamente leves e simples para quem deseja hospedar seu próprio servidor Git com baixo consumo.

GIT VS. GITHUB

CARACTERÍSTICA	GIT	GITHUB
O que é?	Software (Ferramenta)	Serviço (Plataforma)
Onde vive?	Instalado localmente no seu PC	Hospedado na nuvem (Internet)
Função	Gerenciar o histórico de arquivos	Hospedar repositórios e colaborar
Interface	Linha de comando ou Apps GUI	Interface Web e Desktop
Internet	Não precisa para uso local	Essencial para sincronizar

BENEFÍCIOS DE SE USAR GIT E GITHUB

Segurança e Backup

Nunca mais perca seu trabalho por falhas no computador. Seus arquivos estão salvos na nuvem.

Trabalho em Equipe

Várias pessoas podem editar o mesmo projeto sem sobrescrever o trabalho uma da outra.

Histórico Completo

Saiba exatamente quem mudou o quê, quando e por quê. Rastreabilidade total.

Experimentação Livre

Crie "branches" para testar ideias sem medo de quebrar a versão principal do projeto.

Portfólio Profissional

O GitHub serve como um currículo vivo para recrutadores e parceiros técnicos.

Qualidade de Código

Revisões de código (Code Reviews) facilitam a detecção de erros e melhoram o aprendizado.

INSTALANDO O GIT

Primeiro passo: instalar, abrir o terminal e confirmar que o Git responde.

Windows

1. Baixe o Git for Windows
2. Instale com as opções padrão
3. Use Git Bash, PowerShell ou terminal

Verificação:
$ git --version

Linux

Ubuntu/Debian:
$ sudo apt update
$ sudo apt install git

Outras distros usam o gerenciador de pacotes correspondente.

Verificação:
$ git --version

macOS

Opção comum com Homebrew:
$ brew install git

Também é possível instalar pelas Command Line Tools da Apple.

Verificação:
$ git --version

Objetivo: ao final, o comando git --version deve retornar a versão instalada.

CONFIGURAÇÃO BÁSICA DO GIT

Essas informações identificam seus commits e ajustam o padrão de novos repositórios.

```
$ git config --global user.name "Seu Nome"
$ git config --global user.email
"seu@email.com"
$ git config --global init.defaultBranch
main

# Conferir configurações
$ git config --list
```

user.name

Nome que aparecerá como autor dos commits.

user.email

E-mail associado aos commits e ao GitHub.

init.defaultBranch main

Faz novos repositórios começarem na branch main.

O --global vale para todos os repositórios daquele usuário nessa máquina.

AUTENTICAÇÃO COM GITHUB VIA SSH

Use um par de chaves para conectar seu computador ao GitHub sem digitar senha a cada operação.

1. Gerar
ssh-keygen -t ed25519 -C "seu@email.com"

→

2. Agente
ssh-add ~/.ssh/id_ed25519 varia por SO

→

3. GitHub
Cadastrar o conteúdo do id_ed25519.pub

→

4. Testar
ssh -T git@github.com

```
$ ssh-keygen -t ed25519 -C "seu@email.com"
$ eval "$(ssh-agent -s)"
$ ssh-add ~/.ssh/id_ed25519
$ cat ~/.ssh/id_ed25519.pub
$ ssh -T git@github.com
```

id_ed25519	chave privada · nunca compartilhar
id_ed25519.pub	chave pública · cadastrar no GitHub

Observação: no Windows, Linux e macOS o fluxo é o mesmo, mas a ativação do ssh-agent pode mudar.

OS DOIS CAMINHOS INICIAIS DO GIT



Caminho 1: Construindo do Zero

Você tem uma pasta nova no seu PC e quer começar a rastrear as mudanças nela.

git init



Caminho 2: Projeto Existente

O projeto já existe na internet (GitHub) e você quer baixar uma cópia completa.

git clone

CAMINHO 1

Construindo do Zero: git init

Situação: Você acabou de criar uma pasta no seu computador para um projeto totalmente novo. Não existe nada na internet ainda.

A Analogia (O Lote Vazio): Você tem um terreno vazio. Ao digitar git init, você contrata o "assistente" (Git). Ele entra, coloca um crachá e diz: "Pronto, a partir de agora esta sala é a sua Mesa de Trabalho e eu vou vigiar tudo".

Próximos passos: Criar arquivos → git add → git commit → Alugar galpão na nuvem → git push.

"Transformando uma pasta comum em um repositório oficial."



```
$ git init
```

CONECTANDO AO MUNDO: O PRIMEIRO PUSH

O Problema: Se você começou com git init, o Git sabe vigiar sua pasta, mas ele ainda não sabe onde fica o seu depósito na internet (GitHub).

A Solução: Você precisa informar o endereço do seu repositório remoto.

```
$ git remote add origin [URL_DO_GITHUB]
$ git push -u origin main
```

Analogia: É como dar o endereço do galpão alugado para o caminhoneiro. Agora ele sabe exatamente para onde levar as suas caixas lacradas.



Ponto de Conexão

"Criando a ponte entre o seu computador e a nuvem."

CAMINHO 2

Entrando em um Projeto Existente: git clone

Situação: O projeto já está na internet (GitHub). Pode ser de um colega, um projeto open-source ou um trabalho que você começou em outro PC.

A Analogia (O Teletransporte): O comando git clone é pura mágica. Você aponta para o depósito na internet e diz "Clone!".
O Git copia a estrutura inteira: baixa as caixas lacradas, contrata o assistente e monta a sua Mesa de Trabalho completinha.
Resultado: Você senta e já está tudo lá, pronto para editar e colaborar imediatamente.



```
$ git clone [URL]
```

"Uma cópia perfeita do repositório remoto para o seu computador."



REGRA DE OURO

Quem faz git clone
NÃO precisa fazer git init!

O clone já traz o escritório montado e o assistente contratado. Fazer os dois na mesma pasta é como contratar dois pedreiros para construírem a mesma parede – vai dar confusão.

A JORNADA DO ARQUIVO

Entender o Git é como entender um processo de logística. Vamos acompanhar a vida de um arquivo desde o seu nascimento até o servidor na nuvem e o seu retorno.



PASSO 0
O Toque Mágico
(git init)

PASSO 1
Mesa de Trabalho
(Working Dir)

PASSO 2
Caixa Aberta
(Staging Area)

PASSO 3
Caixa Lacrada
(Local Repo)

PASSO 4
O Caminhão
(Remote Repo)

PASSO 5
Trazendo Caixas
(git pull)

PASSO 6
Abrindo Antigas
(git checkout)

ETAPA 0

O Toque Mágico: git init

Antes de mais nada, você tem apenas uma pasta comum no seu computador. O Git não sabe que ela existe.

```
$ git init
```



O que ele faz: É como contratar um assistente de organização.
Você diz: "A partir de agora, esta sala é oficial!".
Ele transforma a sua pasta comum na sua Mesa de Trabalho (Working Directory) vigiada pelo Git.

"O assistente senta no canto da sala com um bloquinho de notas, pronto para observar tudo."

ETAPA 1

Trabalhando na sua Mesa

Com o assistente na sala, a sua Mesa de Trabalho (Working Directory) está pronta.

É aqui que a bagunça acontece: Você cria textos, escreve códigos e apaga arquivos.

O Git está apenas observando de longe, sem interferir no que você está espalhando pela mesa.



UNTRACKED **MODIFIED**

"O Git observa cada movimento, mas ainda não salvou nada oficialmente."

ENXERGANDO A MESA: GIT STATUS

O comando git status é o seu melhor amigo. Ele é como pedir um relatório instantâneo para o seu assistente sobre o que está acontecendo na sala.

```
Terminal - bash

$ git status
```

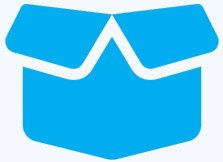
Dica: Use sempre antes de um add e antes de um commit para ter certeza do que está fazendo.

ETAPA 2

Colocando na Caixa Aberta: git add

Quando você termina uma parte do trabalho, precisa começar a empacotar. Você usa o comando git add.

```
$ git add .
```



O que ele faz: Ele pega aquele arquivo pronto da sua mesa e o coloca dentro de uma caixa de papelão aberta, chamada Staging Area (Área de Preparação).

Nota: A caixa ainda está aberta! Se você quiser colocar mais arquivos ou tirar algum antes de fechar, ainda dá tempo.

"A Staging Area é onde você seleciona o que vai entrar na próxima foto do projeto."

ETAPA 3

Lacrando a Caixa: git commit

Quando a caixa (Staging Area) tem tudo o que você precisa guardar, você usa o comando git commit.

```
$ git commit -m "mensagem"
```

O que ele faz: Ele fecha a caixa com fita adesiva, tira uma "foto" dela e cola uma etiqueta descrevendo o conteúdo.
Essa caixa lacrada é guardada na sua garagem segura: o Local Repo (Repositório Local).
Nota: Agora o pacote está salvo no histórico do seu computador, mas ninguém mais no mundo consegue vê-lo ainda.



Local Repo

Sua garagem segura de versões.

CONSULTANDO O HISTÓRICO: GIT LOG

O git log permite que você veja todas as "caixas lacradas" (commits) que já foram guardadas na sua garagem local.

```
$ git log

commit a1b2c3d4e5f6...
Author: Desenvolvedor
Date: Mon May 20 14:00:00 2024 -0300
```

É a prova documental de tudo o que foi feito no projeto desde o primeiro dia.

ETAPA 4

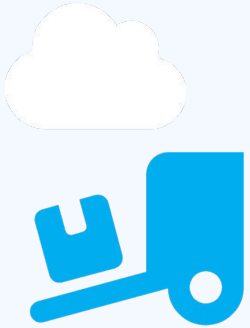
Enviando o Caminhão: git push

Você já tem várias caixas lacradas na sua garagem (Local Repo) e agora quer mandar isso para a internet.

```
$ git push
```

O que ele faz: Ele empurra as suas caixas lacradas para um caminhão que as leva até o depósito na nuvem.

Este depósito remoto é o GitHub. Agora, o seu trabalho está seguro na internet e pronto para o time colaborar.



"As caixas saem da sua garagem local e vão para o servidor remoto."

ETAPA 5

Trazendo Caixas do Depósito: git pull

Imagine que um colega de equipe mandou caixas novas lá para o depósito na nuvem (Remote Repo). Você precisa do trabalho dele para continuar o seu.

```
$ git pull
```

O que ele faz: Ele vai até a internet, puxa essas caixas novas e as despeja direto na sua Mesa de Trabalho (Working Directory).

Isso atualiza seus arquivos locais para que você possa trabalhar com a versão mais recente do projeto.



"Puxando as atualizações da nuvem direto para a sua mesa."

ETAPA 6

Abrindo Caixas Antigas: git checkout

Você se arrependeu de uma mudança ou quer apenas olhar como estava um arquivo ontem. Você usa o comando git checkout.

```
$ git checkout [arquivo]
```

O que ele faz: Ele vai até a sua própria garagem (Local Repo), pega uma daquelas caixas antigas que você já tinha lacrado, abre e coloca os arquivos de volta na sua Mesa de Trabalho.

Isso restaura o estado do arquivo exatamente como ele estava naquele momento do passado.



"Recuperando versões salvas na sua garagem local."

RESUMO: OS DOIS CAMINHOS INICIAIS

Caminho 1: Começando do Zero

git init
Transforma a pasta atual em repositório oficial.

git add .
Coloca tudo na caixa aberta (Staging Area).

git commit -m "..."
Lacra a caixa e salva na garagem local.

git remote add origin [URL]
Informa o endereço do GitHub (uma vez).

git branch -M main
Garante o nome da branch principal.

git push -u origin main
Envia e cria ligação permanente.

Caminho 2: Pegando Projeto Pronto

git clone [URL]
Copia tudo da internet. Já faz o init e configura o remoto.

cd [nome-da-pasta]
Entra na pasta do projeto para começar.

git add .
Coloca modificações na caixa aberta.

git commit -m "..."
Lacra a caixa localmente.

git push
Manda de volta para a internet (simples e direto).

Regra de Ouro: Quem faz CLONE não precisa fazer INIT!